

Prac 2 - Simple Database Access

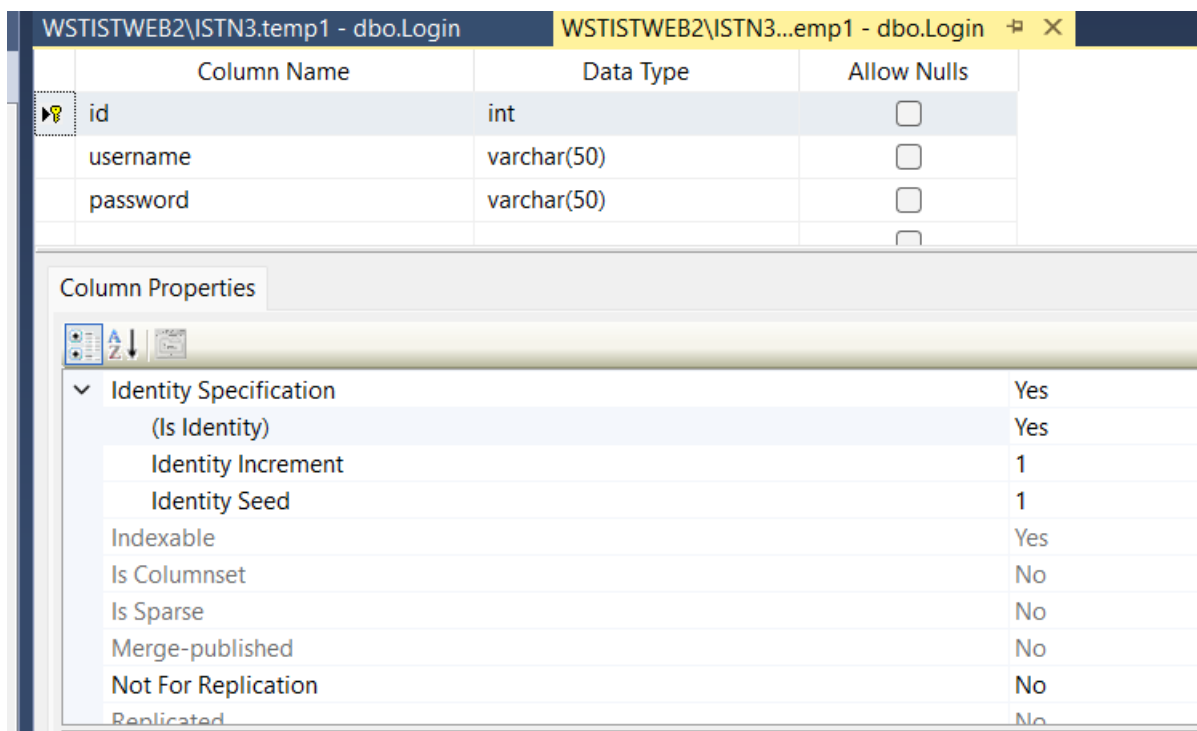
This prac is a continuation of the prac on menu design – the menu interface created in the previous prac will be used to demonstrate a trivial database interaction.

The interaction that will be modelled is the development of the log-in functionality.

The user credentials will be obtained from the database.

Part One – Creating the Database

Create a database table in your individual database on the Sql Server database. This can be done by making use of the Sql Server Management Studio client application. Create a table named **Login** and design the table so that it contains the following fields.

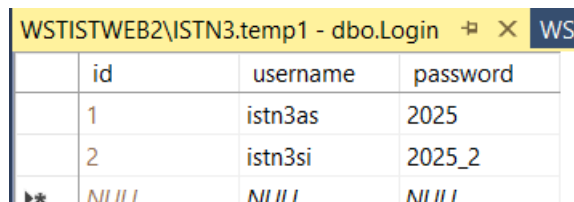


Column Name	Data Type	Allow Nulls
id	int	☐
username	varchar(50)	☐
password	varchar(50)	☐

Column Properties	
Identity Specification	Yes
(Is Identity)	Yes
Identity Increment	1
Identity Seed	1
Indexable	Yes
Is Columnset	No
Is Sparse	No
Merge-published	No
Not For Replication	No
Replicated	No

Note the Primary Key and the Identity Specification so that each new user entry has a unique identifying number. This is actually not necessary – but for the sake of convenience – we'll set up the table this way.

The next step is to capture username and password data via the User Menu option from the main application so that these users will have access to the application. A sample set of usernames and passwords is shown below (captured via the system).



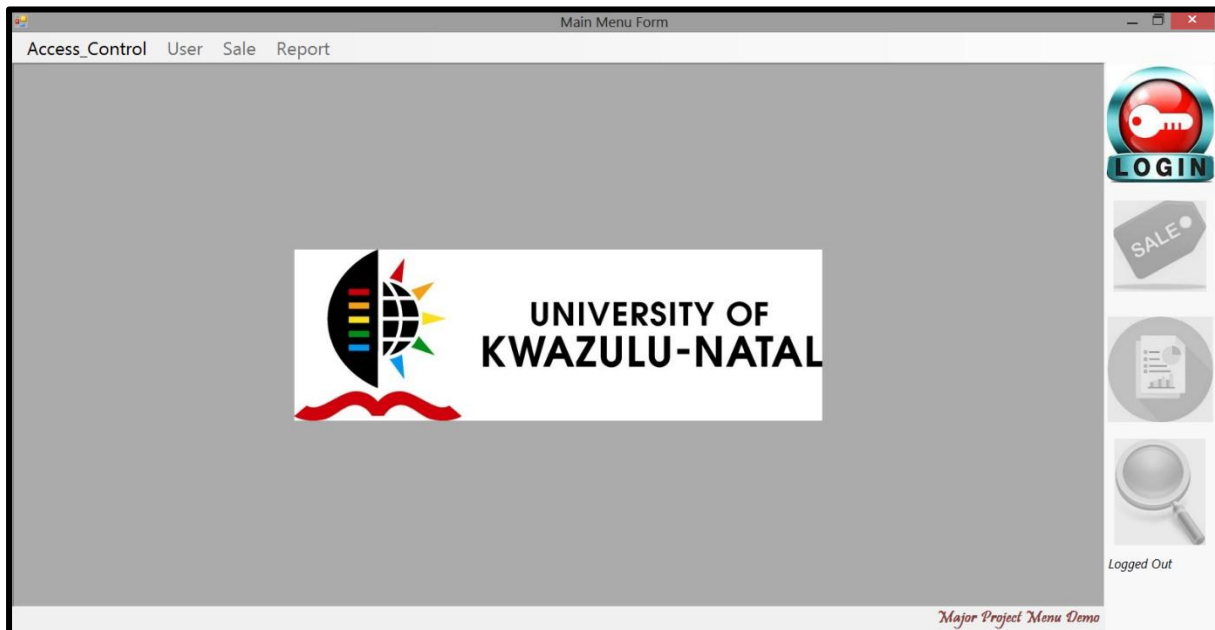
id	username	password
1	istn3as	2025
2	istn3si	2025_2
NULL	NULL	NULL

Note that the UserNo is automatically assigned by the database and any new user added will have a userNo that is an increment of the previous userNo.

And so the database is ready for interaction with the main application.

Part Two – The Main application and the Database

The main interface looks like....



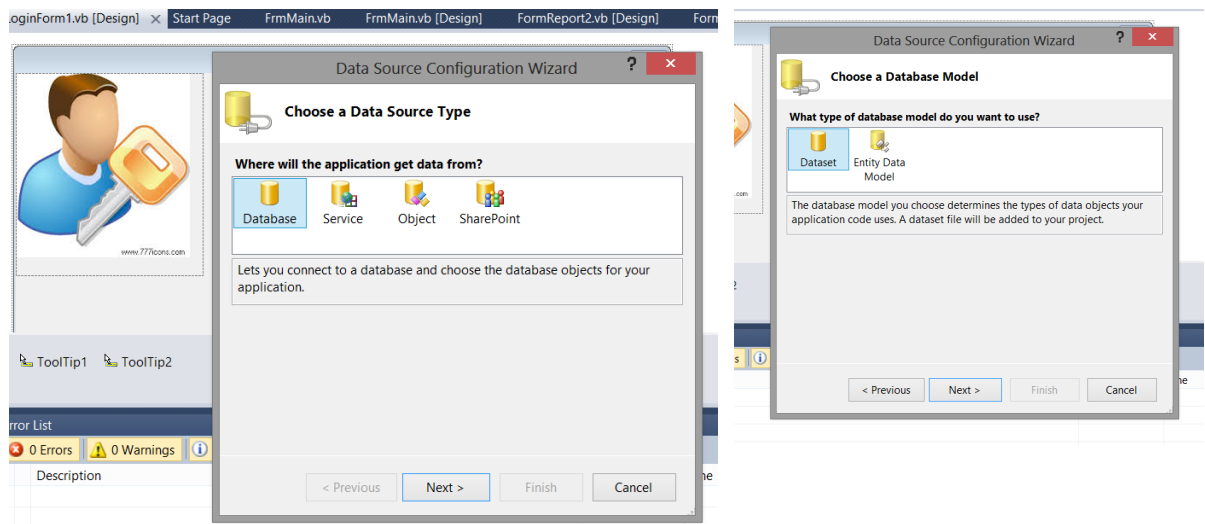
The processing sequence entails requires that the user engages the “Access Control” part of the menu and activates the log-in screen. Once this occurs, the user’s credentials are then validated against the database entries in the table named **Login**. If the user name and password is valid, then the remainder of the menu options become available.

The Database Interaction

In this part of the prac exercise, you will have to make use of the suite of database objects so that access to the Sql Server database is achieved. The data architecture of the .Net framework relies heavily on a **disconnected dataset**. This is basically a memory based version of the Sql Server database table. In order to obtain the memory based version of the Sql Server data, you will have to invoke the dataset object that starts a wizard and guides you through the process of configuring the type of data access that you will require. In the case of the current application, we would like to send the username and password as input parameters into a Sql statement that returns those rows in the database that have an exact match to the username and password.

The following screenshot will illustrate the setting up of access to a Sql Server Database.

1. Create a new data source (via the menu – Project>>Add New Data Source>>Dataset....

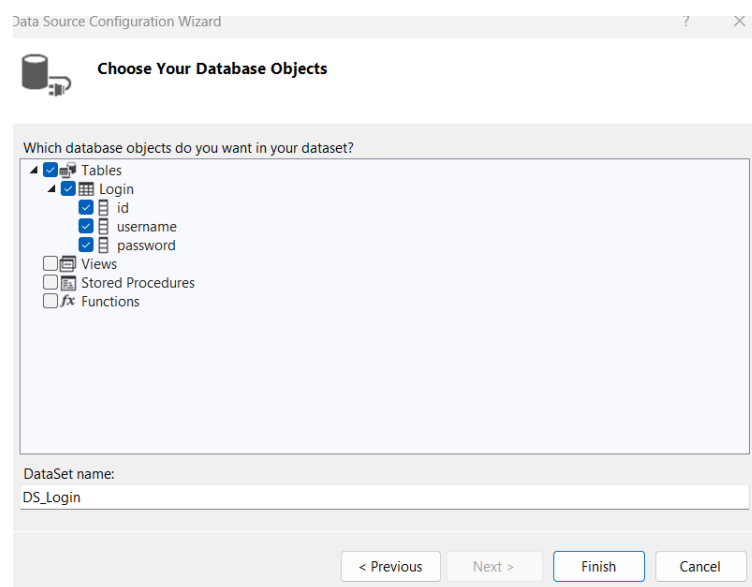


2. Now establish a new connection to your database – note **146.230.177.46\ISTN3**

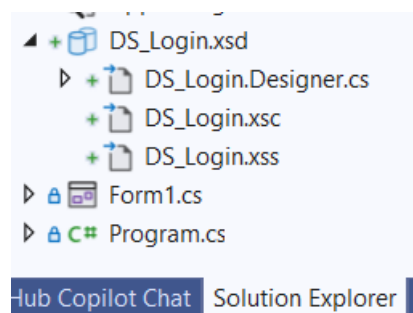
The image shows the 'Add New Connection' dialog box for Microsoft SQL Server (SqlClient). The 'Data source' is set to 'Microsoft SQL Server (SqlClient)'. The 'Server name' is '146.230.177.46\istn3'. The 'Log on to the server' section has 'Authentication' set to 'SQL Server Authentication'. The 'User name' is 'temp1' and the 'Password' is masked with dots. The 'Encrypt' option is set to 'Mandatory (True)'. The 'Trust Server Certificate' checkbox is checked. The 'Connect to a database' section has 'Select or enter a database name:' selected, with 'temp1' entered in the dropdown. The 'Test Connection' button is highlighted.

Note: The name of your database corresponds to **your** username

3. Navigate through the wizard until it gives you the option of choosing the database table that you would like to interact with. You can choose to interact with all the rows from this table, or simply just a subset of the rows (such as the rows that correspond to the username and password). The disconnected architecture requires that you first provide details of the table with which you intend to interact with. It uses this information to set up the schema (design) for the memory resident version of the database table. This is named a dataset. ***The dataset is a memory resident version of the data in the database table. In order to achieve this functionality, the wizard guides you through the creation of a dataset class which is then made available in your toolbox as database interaction objects when you compile the application.*** The naming convention that is normally adopted is to use the letters **DS** as a prefix to the name of the dataset class. In the case of the current application, the dataset class is named DS_Login (DataSet for the Login Table) so that the name is also reflective of the database table from which the data is obtained.



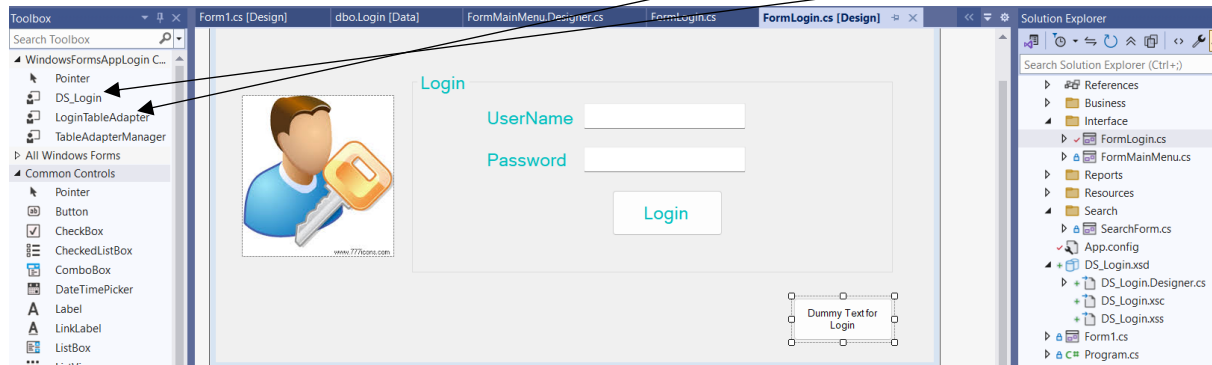
4. At this stage VS2022 will provide a set of database interaction classes that is made available via the Solution Explorer.



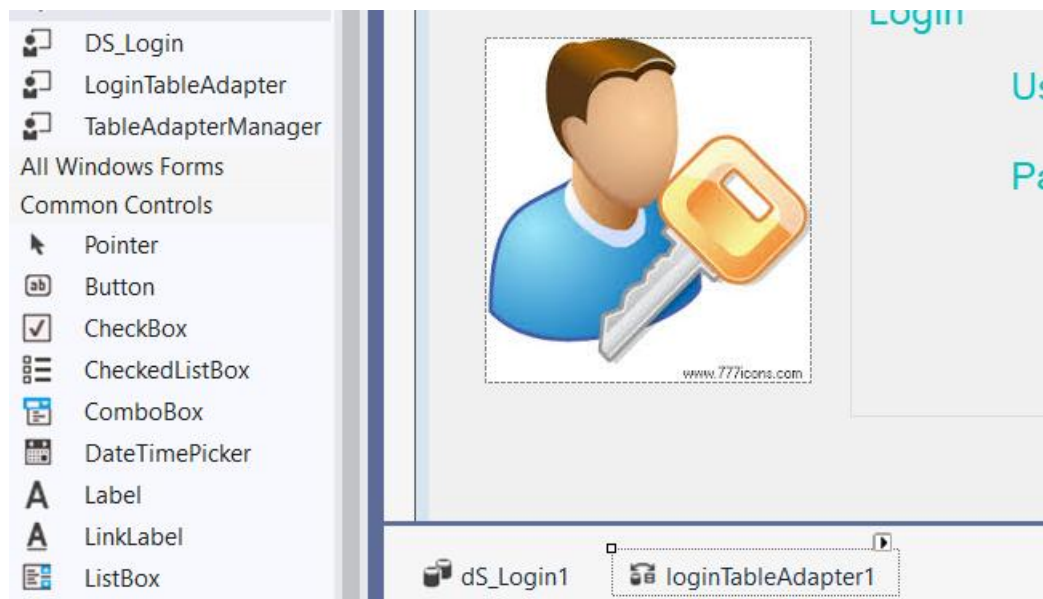
5. The next step is to make the new schema (classes) that have just been created by the wizard to become available so that you can use them in your application. This is achieved by choosing the following options from the VS IDE menu:

Build>>Rebuild...YourApplication

Now if you scroll the toolbox back up to the top, you should see 3 data access classes added to the toolbox. The most important of these is the **TableAdapter** and **Dataset** classes.



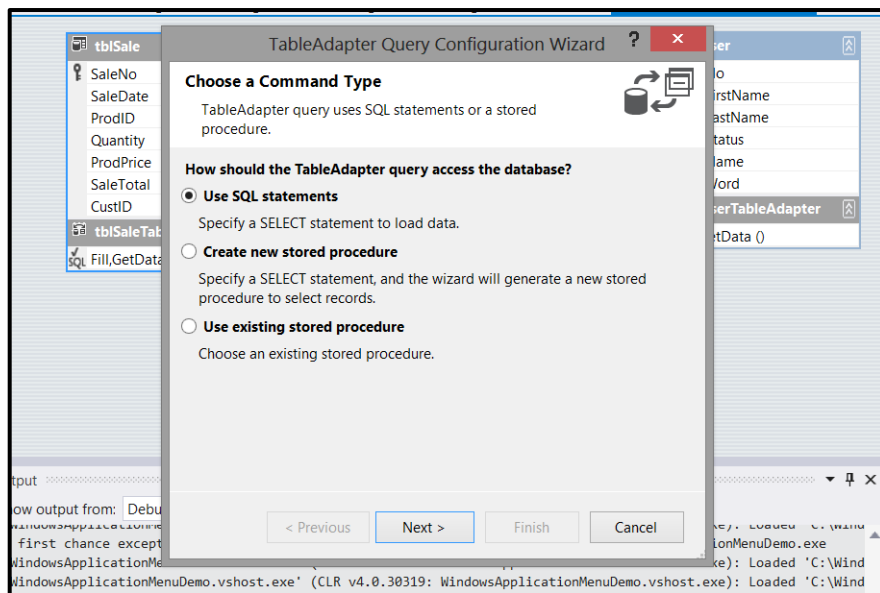
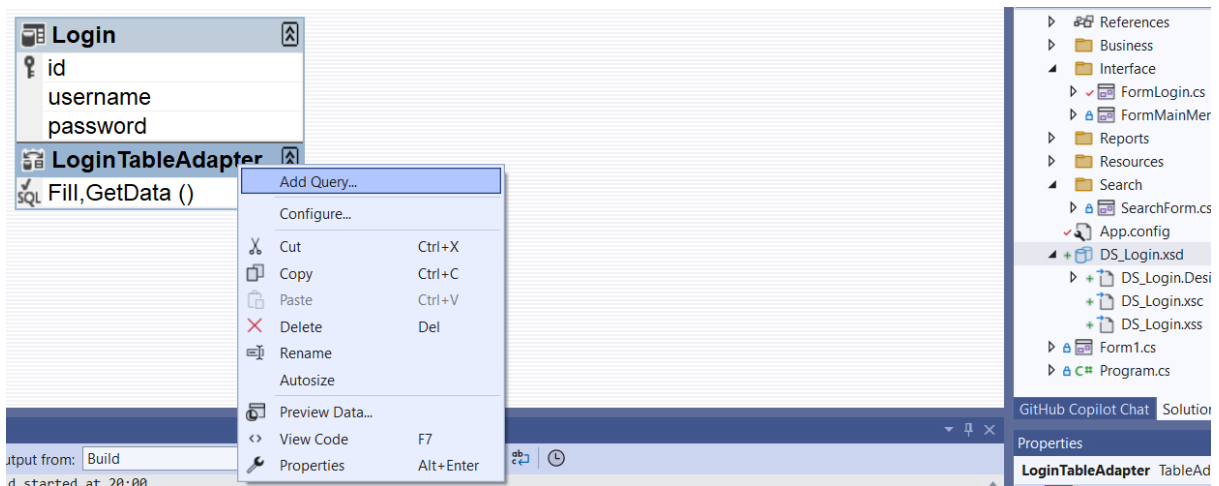
6. Firstly, **instantiate the Dataset** class and the TableAdapter class by dragging these objects and dropping it onto your form.



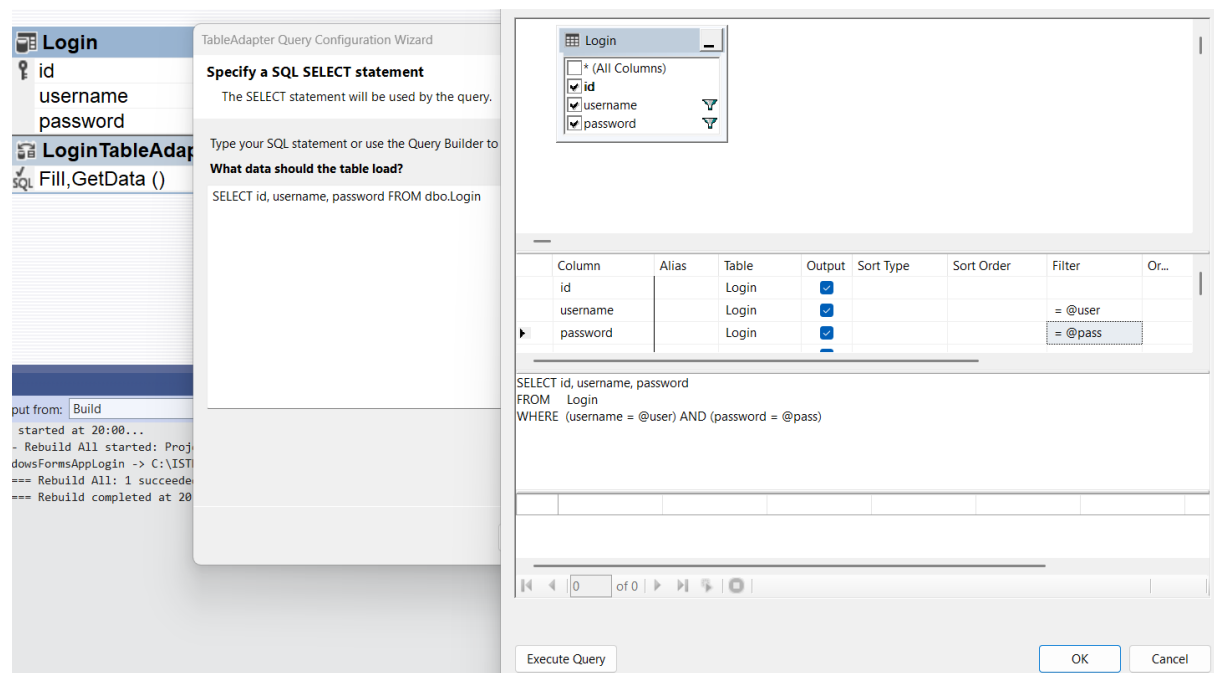
These objects are used to enable database access; the idea is that you use the loginTableAdapter1 object to fill data into the dS_Login1 object. By default, the TableAdapter class has a method named "**Fill**" that simply retrieves all the records from the database table named tblUser and populates the dataset named DSLogin. However, if you are not happy with this default arrangement, then you can always re-configure the TableAdapter and generate a new method that basically uses Sql to retrieve data according to your requirements. However, since we want to look for a specific set of records and we don't want ALL the records from the database, we can

go to the DS_Login class in the Solution Explorer and add a new method that will only retrieve the records that we are looking for.

- For the purpose of the current exercise, we will edit **re-configure the query of the TableAdapter** so that it makes use of a filter – that is we will generate a filtered Sql Query so that only those rows that match our criteria will be returned. This is achieved by “playing around” with the Query Builder until we achieve the desired result. Double click the DS_login class shown as an .xsd file in the solution explorer, to open up the class designer; right click and add a new query *in the dataset designer window* such as the one shown below



- You will then encounter the Sql Query Builder. Invoke the Query Builder so that you are presented with an interface that allows you to start generating customised Sql statements. The Query Builder interface is quite convenient in enabling to generate complex Sql Queries without actually worrying too much about the Sql syntax.



Customising the Sql with a parameter filter: This is achieved by typing in a parameter value in the filter column. By convention, all parameter values have a “@” as a pre-fix.

- The logic that is aligned to our application is that we would like to retrieve those records from the database that match the username and password entered. If there is a match, then a single record should be returned and stored in the memory resident dataset object. If there is no match, then the memory resident dataset will have no records – i.e. a record count of zero.

The parameter names that you use is arbitrary – as soon as you have included the parameter based filters, the Query Builder generates the corresponding Sql statement and displays it as:

```
SELECT UserName, Password
FROM Login
WHERE (userName = @user) AND (passWord = @pass)
```

- The TableAdapter has now been configured according to your “Sql_expectations” – the QueryBuilder then customises its Fill method so that it obtains data according to the Sql statement that has just been created...this customisation is reflected in the signature of the fill method which should contain a parameter list that requires the input of username and password. This customised Fill method is illustrated below (shown in the TableAdapter designer)

11. The next step entails invocation of the FillBy method of the TableAdapter object so that the data is retrieved from the database by the TableAdapter and filled into the dataset. This event should occur as soon as the user has entered his/her credentials (username and password) and clicked the log-in button. The code that will control the event of clicking the log-in button (I've changed the name to Login_Ok) is shown below...when the OK button is clicked...

```
taUser1.FillBy(dsLogin1.tblUser, txtUserName.Text, txtPassword.Text);
if (dsLogin1.tblUser.Rows.Count > 0)
    MessageBox.Show("WELCOME to the MAIN SYSTEM!! ");
    Form frm = (Form)Application.OpenForms["FormMainMenu"];
    MenuStrip ms = (MenuStrip)frm.Controls["menuStrip1"];
    ms.Items["saleMenu"].Enabled = true;
    ms.Items["reportMenu"].Enabled = true;
    ms.Items["searchMenu"].Enabled = true;
    ToolStripMenuItem ti = (ToolStripMenuItem)ms.Items["Access"];
    ti.DropDownItems["logoutMenu"].Enabled = true;
    ti.DropDownItems["loginMenu"].Enabled = false;
    ToolStrip ts = (ToolStrip)frm.Controls["toolStrip1"];
    ts.Items["toolStripLogin"].Enabled = false;
    ts.Items["toolStripSale"].Enabled = true;
    ts.Items["toolStripSearch"].Enabled = true;
    ToolStripTextBox tuser = (ToolStripTextBox)ms.Items["UserNameTxt"];
    tuser.ForeColor = Color.Green;
    DataRow dr = dsLogin1.tblUser.Rows[0];
    tuser.Text = "Logged in as " + dr["UserFirstName"] + " " + dr["UserLastName"];
}
}
if (!(found))
    MessageBox.Show("Error in Login");
}
else
    {
        MessageBox.Show("Error in Login");
    }
```

A quick discussion of the code:

The statement that uses the TableAdapter to fill up the dataset with the results of the parameterised query is shown below:

```
taUser1.FillBy(dsLogin1.tblUser, txtUserName.Text, txtPassword.Text);
```

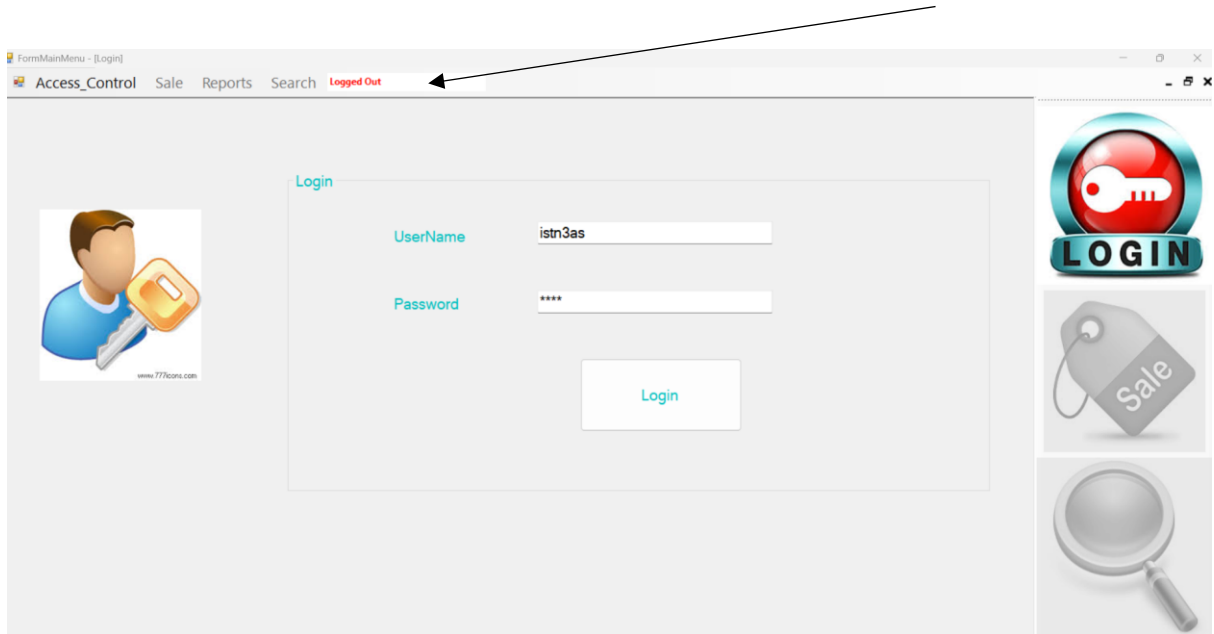
Basically, the fill method is invoked and used to fill the dataset (that's already instantiated as a memory resident object) with the results of the Sql query generated by the fill method. The Sql query is filtered according to the parameter values that are obtained from the UserName textbox and the Password textbox of the log-in screen.

Once the fill has taken place – you can do a check to see if the dataset actually did receive any records – if it did – then the user's username is used to issue a welcome message. In order to access the rows of the dataset, you can make use of the row (number) and cell (number) format to actually access data from the dataset.

The remainder of the code is simply used to manage the user interface.

Extra Challenge

Add a textbox on the menu that informs the user that s/he is either logged out or logged in.



If the user is logged in then display the name of the logged-in user.

